

Manual Técnico

Sistema Lunaria - Gestión de Inventario y Ventas

Versión 1.0

Fecha de creación: Marzo 2026

Sistema: Lunaria v2.0

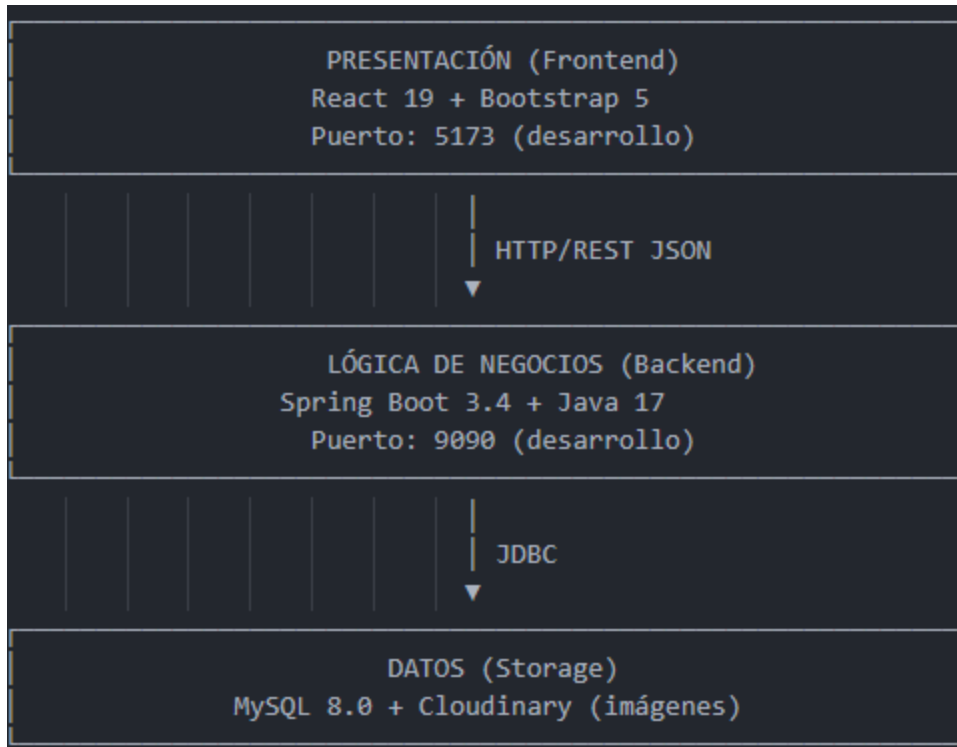
Índice

1. Arquitectura del Sistema
 2. Estructura del Proyecto
 3. Tecnologías Utilizadas
 4. Configuración de Componentes
 5. Diagrama de Arquitectura
 6. Base de Datos
 7. Seguridad
 8. APIs y Endpoints
 9. Despliegue
 10. Mantenimiento
-

1. Arquitectura del Sistema

1.1 Modelo de Arquitectura

El sistema Lunaria sigue una arquitectura de tres capas (3-Tier):



1.2 Tipo de Arquitectura

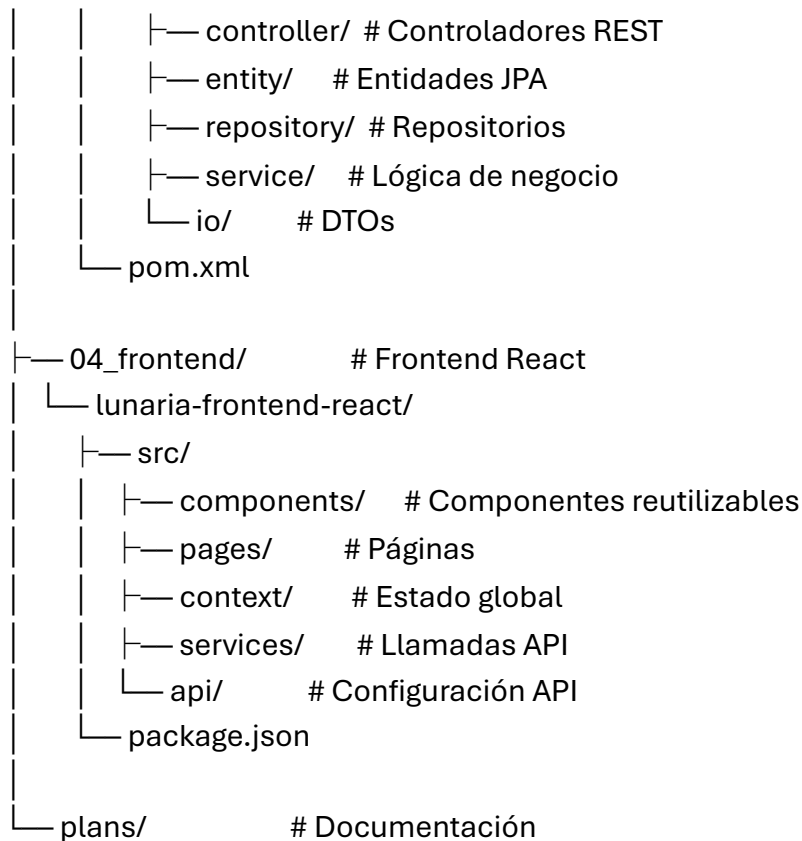
- Cliente-Servidor: Aplicación web responsive
- REST API: Comunicación mediante servicios web RESTful
- Microservicios monolíticos: Backend modular

2. Estructura del Proyecto

2.1 Estructura General

Lunaria/

```
|— 02_database/          # Scripts de base de datos
|   |— lunaria_database.sql  # Esquema de BD
|   |— create_admin_user.sql # Usuario inicial
|
|— 03_backend/          # Backend Spring Boot
|   |— lunaria-backend-springboot/
|       |— src/main/java/
|           |— com/santiago_rachen/lunaria_backend_springboot/
|               |— config/  # Configuraciones
```



2.2 Estructura del Backend

```
src/main/java/com/santiago_rachen/lunaria_backend_springboot/
```



```

├── FavoriteController.java    # Favoritos
├── ItemController.java       # Productos
├── SaleController.java       # Ventas
├── StockController.java      # Inventario
├── UserController.java       # Usuarios
├── entity/
│   ├── BrandEntity.java
│   ├── CategoryEntity.java
│   ├── FavoriteEntity.java
│   ├── ItemEntity.java
│   ├── SaleEntity.java
│   ├── SaleItemEntity.java
│   ├── StockMovement.java
│   └── UserEntity.java
├── repository/
│   ├── BrandRepository.java
│   ├── CategoryRepository.java
│   ├── FavoriteRepository.java
│   ├── ItemRepository.java
│   ├── SaleEntityRepository.java
│   ├── SaleItemEntityRepository.java
│   ├── StockMovementRepository.java
│   └── UserRepository.java
├── service/
│   ├── BrandService.java
│   ├── CategoryService.java
│   ├── FavoriteService.java
│   ├── ItemService.java
│   ├── SaleService.java
│   ├── StockService.java
│   └── UserService.java
├── service/impl/
│   ├── BrandServiceImpl.java
│   ├── CategoryServiceImpl.java
│   ├── FavoriteServiceImpl.java
│   ├── ItemServiceImpl.java

```

```
|   ├── SaleServiceImpl.java
|   ├── StockServiceImpl.java
|   └── UserServiceImpl.java
|── io/
|   ├── AuthRequest.java
|   ├── AuthResponse.java
|   ├── ItemRequest.java
|   ├── ItemResponse.java
|   └── ...
|── filter/
|   └── JwtRequestFilter.java
|── util/
|   └── JwtUtil.java
```

3. Tecnologías Utilizadas

3.1 Backend

Tecnología	Versión	Propósito
Spring Boot	3.4.4	Framework principal
Java	17	Lenguaje de programación
Spring Security	6.x	Seguridad
Spring Data JPA	3.x	Acceso a datos
MySQL Connector	8.x	Driver MySQL
JWT	0.9.1	Autenticación
Lombok	1.18.x	Reducción de código
Swagger/OpenAPI	3.x	Documentación API
Cloudinary SDK	-	Almacenamiento de imágenes
Brevo SDK	-	Envío de correos

3.2 Frontend

Tecnología	Versión	Propósito
React	19.0.0	Framework UI
Vite	6.2.0	Build tool
Bootstrap	5.3.x	Estilos CSS
React Router	6.x	Enrutamiento
Axios	1.7.x	Cliente HTTP
React Context	-	Estado global

3.3 Infraestructura

Servicio	Propósito	Costo
Railway	Backend + MySQL	\$5/mes
Vercel	Frontend	Gratis
Cloudinary	Imágenes	Gratis (inicio)
Brevo	Emails	Gratis (inicio)

4. Configuración de Componentes

4.1 application.properties

Servidor

server.port=\${SERVER_PORT:9090}

server.servlet.context-path=\${SERVER_SERVLET_CONTEXT_PATH:/api/v1.0}

Base de datos

spring.datasource.url=\${SPRING_DATASOURCE_URL:jdbc:mysql://localhost:3306/lunaria_database}

spring.datasource.username=\${SPRING_DATASOURCE_USERNAME:root}

spring.datasource.password=\${SPRING_DATASOURCE_PASSWORD:}

JPA/Hibernate

```
spring.jpa.hibernate.ddl-auto=${SPRING_JPA_HIBERNATE_DDL_AUTO:validate}
```

```
# JWT
```

```
jwt.secret.key=${JWT_SECRET_KEY:lunaria_secret_key}
```

```
# Cloudinary
```

```
cloudinary.cloud_name=${CLOUDINARY_CLOUD_NAME}
```

```
cloudinary.api_key=${CLOUDINARY_API_KEY}
```

```
cloudinary.api_secret=${CLOUDINARY_API_SECRET}
```

```
# Archivos
```

```
spring.servlet.multipart.max-file-size=20MB
```

```
spring.servlet.multipart.max-request-size=20MB
```

4.2 SecurityConfig.java (Extracto)

```
@Configuration
```

```
@EnableWebSecurity
```

```
public class SecurityConfig{
```

```
    @Autowired
```

```
    private JwtRequestFilter jwtRequestFilter;
```

```
    @Bean
```

```
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
```

```
        http
```

```
            .csrf(csrf -> csrf.disable())
```

```
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers("/login", "/register", "/password-reset/**").permitAll()
```

```
                .requestMatchers("/admin/**").hasRole("ADMIN")
```

```
                .anyRequest().authenticated()
```

```
            )
```

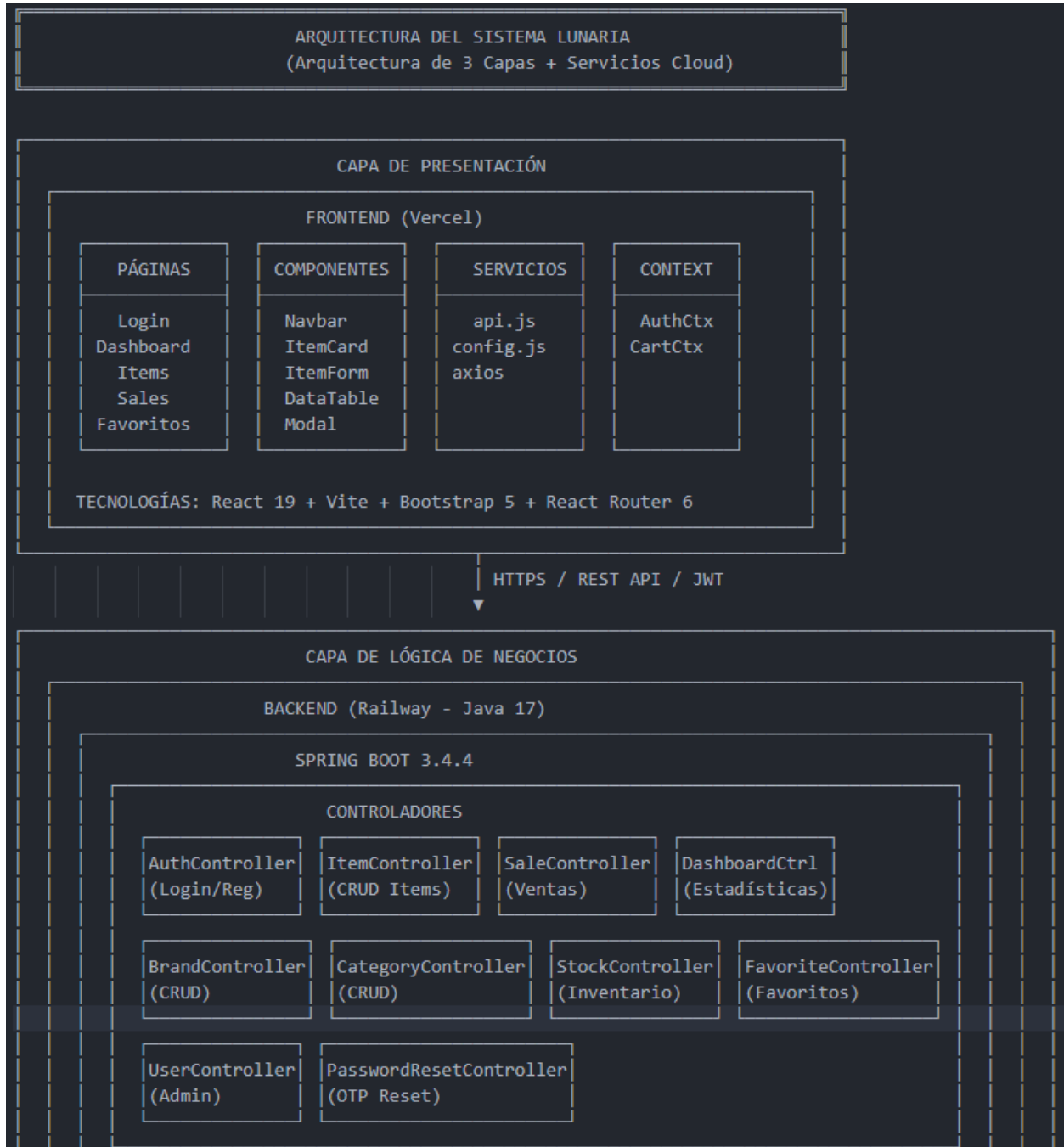
```
            .addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);
```

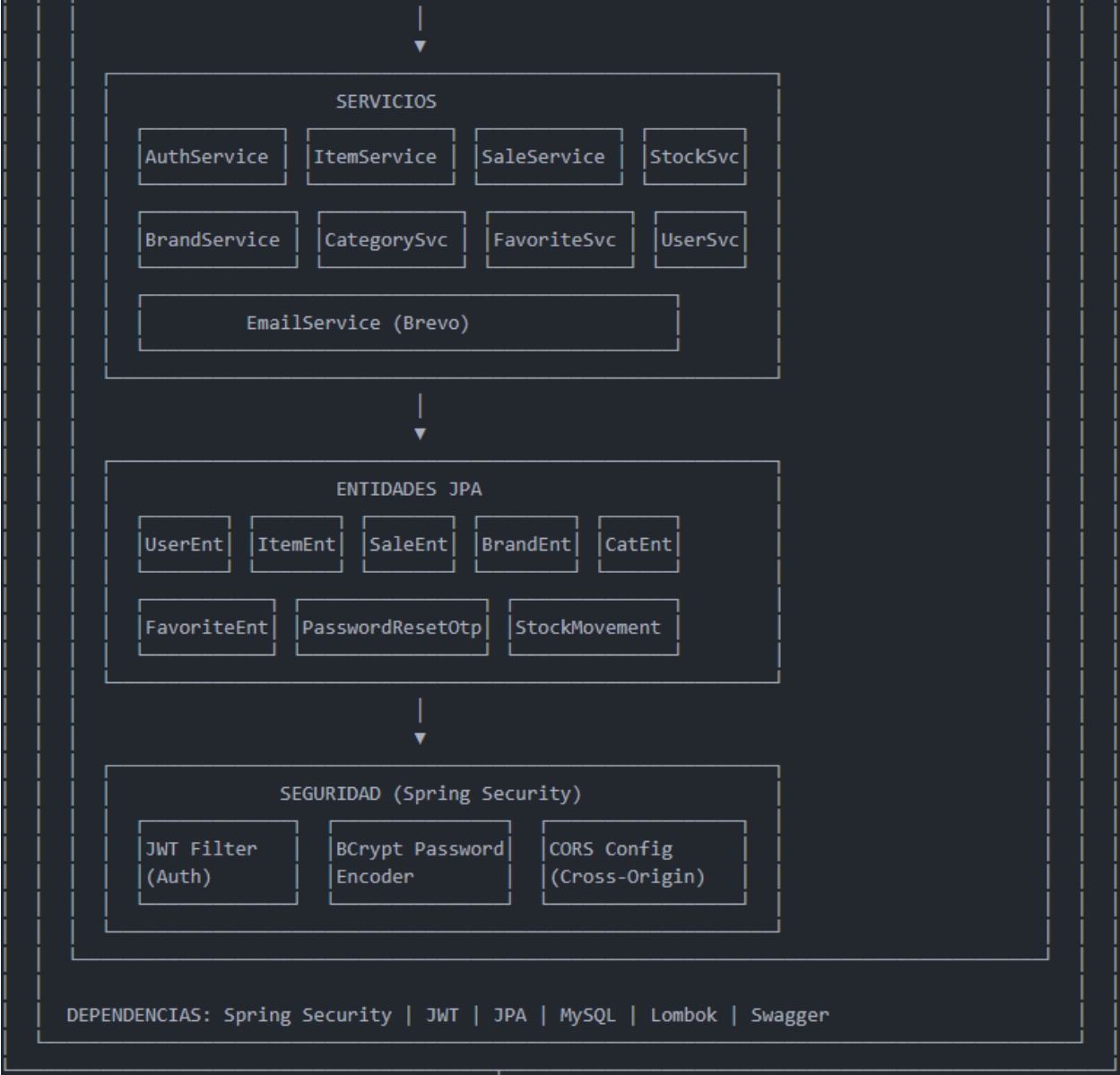
```
        return http.build();
```

```
    }
```

```
}
```

5. Diagrama de Arquitectura





JDBC / REST



CAPA DE DATOS

MYSQL (Railway)

BASE DE DATOS

tbl_users
tbl_category
tbl_brand
tbl_items
tbl_sales
tbl_sale_items
tbl_favorites
tbl_stock_mov
password_reset_otp

Motor: InnoDB
Charset: utf8mb4

CLOUDINARY

ALMACENAMIENTO

Imágenes de
productos

- Optimización
- CDN global
- Transformaciones

SERVICIOS EXTERNOS

RAILWAY
(Infraestructura)

- Backend
- MySQL

BREVO
(Email)

- OTP
- Notificaciones

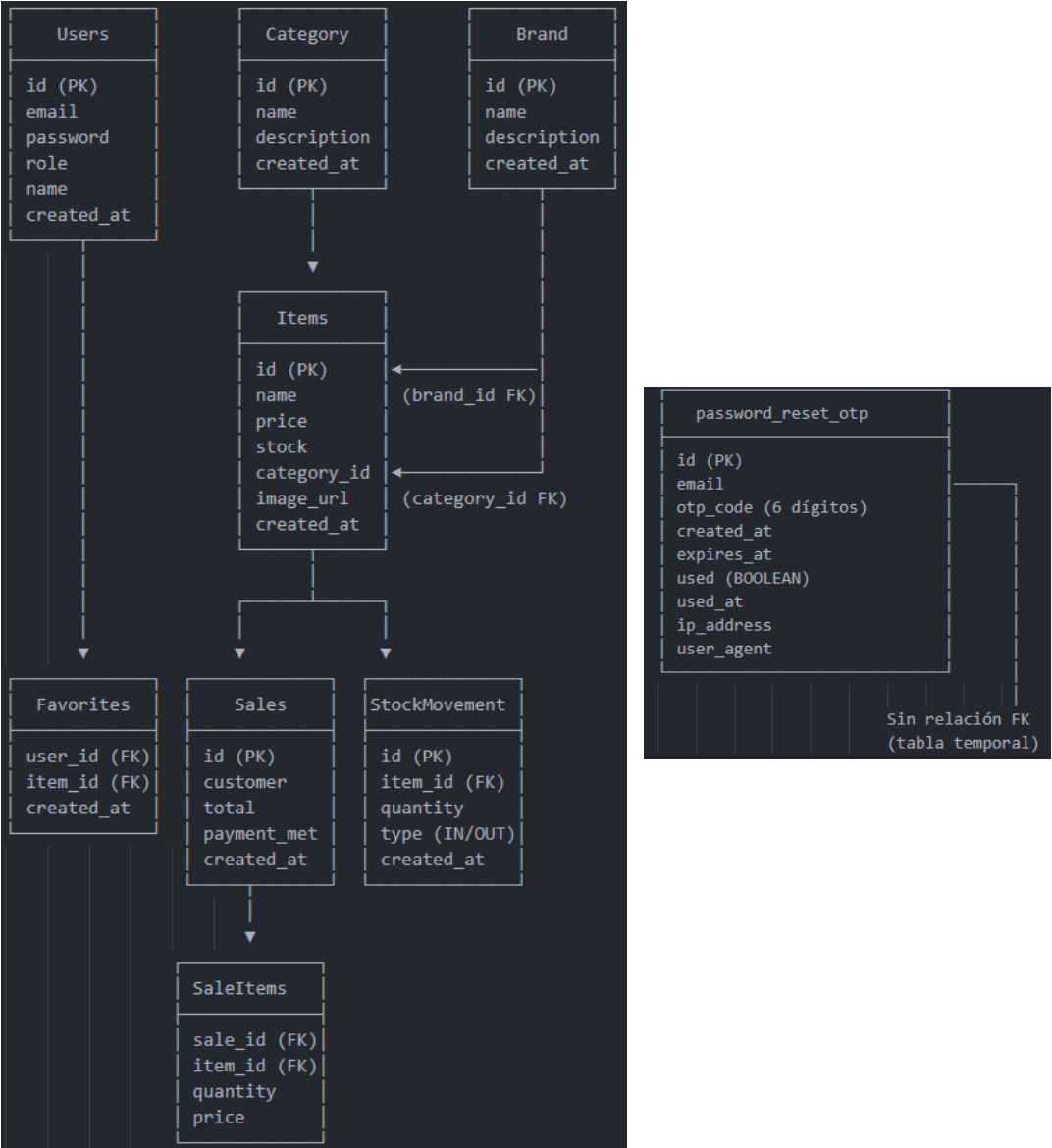
CLOUDINARY
(Imágenes)

- Storage
- CDN

1. USUARIO → FRONTEND (React)
 - ↳ Acciones del usuario (clic, formularios)
2. FRONTEND → BACKEND (API REST)
 - ↳ Peticiones HTTP con JWT Bearer Token
3. BACKEND (Controladores)
 - ↳ Valida request → Llama a Servicio
4. BACKEND (Servicios)
 - ↳ Lógica de negocio → Valida reglas
5. BACKEND (Repositorios)
 - ↳ Consulta/Modifica datos en MySQL
6. RESPUESTA → FRONTEND
 - ↳ JSON con datos o errores
7. SERVICIOS EXTERNOS:
 - ↳ Cloudinary: Subida/Descarga de imágenes
 - ↳ Brevo: Envío de emails (OTP)

6. Base de Datos

6.1 Modelo Entidad-Relación



6.2 Tablas Principales

Tabla	Descripción	Registros típicos
tbl_users	Usuarios del sistema	10-100
tbl_category	Categorías de productos	5-20
tbl_brand	Marcas de productos	10-50
tbl_items	Productos	100-1000
tbl_sales	Registro de ventas	500-10000

tblsaleitems	Items de cada venta	1000-50000
tbl_favorites	Productos favoritos	50-500
tblstockmovements	Historial de inventario	500-10000
passwordresetotp	Códigos OTP recuperación	Por uso

6.3 Tabla de Recuperación de Contraseña (OTP)

Campo	Tipo	Descripción
id	BIGINT	Identificador único (PK)
email	VARCHAR(255)	Email del usuario
otp_code	VARCHAR(6)	Código OTP de 6 dígitos
created_at	TIMESTAMP	Fecha de creación
expires_at	TIMESTAMP	Fecha de expiración (10 min)
used	BOOLEAN	Si el código fue usado
used_at	TIMESTAMP	Fecha de uso
ip_address	VARCHAR(45)	IP del solicitante
user_agent	VARCHAR(500)	Navegador/dispositivo

7. Seguridad

7.1 Autenticación

- Tipo: JWT (JSON Web Tokens)
- Algoritmo: HS256
- Expiración: 24 horas
- Almacenamiento: Frontend (localStorage)

7.2 Autorización

Rol	Permisos
ROLE_ADMIN	CRUD completo, Dashboard, Usuarios
ROLE_USER	Lectura, Ventas, Favoritos

7.3 Medidas de Seguridad

- Contraseñas encriptadas con BCrypt
 - Tokens JWT con expiración
 - Validación de entradas
 - Control de acceso por roles
 - Protección CORS
 - HTTPS en producción
-

8. APIs y Endpoints

8.1 Autenticación

Método	Endpoint	Descripción	Acceso
POST	/login	Iniciar sesión	Público
POST	/register	Registrarse	Público
POST	/password-reset/request	Solicitar código OTP	Público
POST	/password-reset/reset	Restablecer contraseña	Público
POST	/password-reset/resend	Reenviar código OTP	Público

8.2 Productos

Método	Endpoint	Descripción	Acceso
GET	/items	Listar productos	Público
GET	/items/{id}	Ver producto	Público

POST	/admin/items	Crear producto	ADMIN
PUT	/admin/items/{id}	Editar producto	ADMIN
DELETE	/admin/items/{id}	Eliminar producto	ADMIN

8.3 Categorías y Marcas

Método	Endpoint	Descripción	Acceso
GET	/categories	Listar categorías	Público
POST	/admin/categories	Crear categoría	ADMIN
GET	/brands	Listar marcas	Público
POST	/admin/brands	Crear marca	ADMIN

8.4 Ventas

Método	Endpoint	Descripción	Acceso
GET	/sales	Listar ventas	USER
POST	/sales	Crear venta	USER
GET	/sales/latest	Ventas recientes	ADMIN

8.5 Dashboard

Método	Endpoint	Descripción	Acceso
GET	/dashboard	Estadísticas	ADMIN

8.6 Favoritos

Método	Endpoint	Descripción	Acceso
GET	/favorites	Mis favoritos	USER

POST	/favorites/{id}	Agregar favorito	USER
DELETE	/favorites/{id}	Quitar favorito	USER

9. Despliegue

9.1 Backend (Railway)

1. Conectar repositorio de GitHub
2. Railway detecta Spring Boot automáticamente
3. Configurar variables de entorno
4. Deploy automático desde rama main

9.2 Frontend (Vercel)

1. Importar repositorio de GitHub
 2. Framework detectado: Vite + React
 3. Configurar variable VITE_API_BASE_URL
 4. Deploy automático desde rama main
-

10. Mantenimiento

10.1 Tareas de Mantenimiento

Frecuencia	Tarea
Diario	Revisar logs de errores
Semanal	Backup de base de datos
Mensual	Actualizar dependencias
Trimestral	Revisión de seguridad

10.2 Monitoreo

- Railway: Métricas de CPU, memoria, red

- Vercel: Métricas de rendimiento
- Cloudinary: Uso de almacenamiento